Concurrent Pipeline Stages Optimization for Embedded Keyword Spotting

Giacomo Zema*, Valentino Peluso†, Andrea Calimera†, Enrico Macii*
*Interuniversity Department of Regional and Urban Studies and Planning, Politecnico di Torino, Turin, Italy
†Department of Control and Computer Engineering, Politecnico di Torino, Turin, Italy
{giacomo.zema, valentino.peluso, andrea.calimera, enrico.macii}@polito.it

2023 IEEE World AI IoT Congress (AIIoT) | 979-8-3503-3761-7/23/$31.00 ©2023 IEEE | DOI: 10.1109/AIIoT58121.2023.10174531

***Abstract*—Voice interaction has rapidly become the preferred option for human-machine interactions, especially for domotics and wearable applications. In such contexts, the availability of accurate, yet resource-efficient keyword-spotting (KWS) systems for identifying user commands and queries is paramount. In this work, we focus on optimizing the development of KWS pipelines for voice user interfaces running on commercial embedded systems that can be deployed at the edge of the Internet of Things. Specifically, we investigate the joint optimization of the *frontend* and *ConvNet* stages of the KWS information processing pipeline, demonstrating how a holistic approach can significantly improve the quality of results of standard methods that apply optimizations on each stage of the pipeline independently. As the main contributions, we first define and discretize the design space to identify the most significant hyperparameters and then introduce a framework to search the dominant pipeline implementations in the accuracy-latency objective space. Porting and testing the explored solutions on an ARM Cortex-A72 CPU core embedded into a Raspberry Pi 4 board, the collected results show substantial improvements in accuracy (1.8% best-case, 0.82% average) and latency (best-case 27.36%, average 15.32%) compared to state-of-the-art approaches.**

***Index Terms*—Embedded Systems, Keyword Spotting, Convolutional Neural Networks, Speech Signal Processing**

I. Introduction

Keyword Spotting (KWS) is a speech recognition task for the identification of specific commands/queries in spoken language. With the increasing demand for seamless human-machine communications, KWS is a must-have feature for many Internet of Things devices deployed at the edge, like smart assistants for domotics and automotive applications, hearing aids and smartphones, and, in general, any voice-activated application and device [1]–[3]

The implementation of KWS requires significant design efforts to reach satisfactory levels of accuracy. The rise of deep learning (DL) models [4] has contributed to achieving human-level quality yet at the cost of higher computational complexity, which contrasts with the limited resource budget available on consumer electronics. Among the many existing DL-based KWS solutions, only a restricted set of them are compliant with the typical specifications of resource-bounded embedded systems [5]. The most common and widely adopted strategy for embedded KWS consists of a two-stage pipeline with a signal processing frontend followed by a Convolutional Neural Network (ConvNet). The frontend involves handcrafted feature extraction using standard signal processing techniques for audio data, with Mel-Frequency Cepstral Coefficients (MFCCs) as the most commonly adopted. The features generated by the frontend stage are then fed to the ConvNet classifier, which returns the probability that the detected keyword belongs to one of the available labels used during supervised training.

Although other KWS strategies may return superior prediction accuracy, this pipeline offers a good performance trade-off [5], which is paramount when dealing with hardware constraints. Indeed, several previous studies have demonstrated its portability on a wide set of embedded hardware architectures, from low-power high-end CPUs, such as the Cortex-A [6] targeted in this work, to tiny microcontroller units (MCUs) running at a few hundreds of MHz and below [7]. Those achievements are the result of optimizations techniques applied on both the frontend and ConvNet stage [7]–[20] and aimed to reduce the compute and memory workload without (or marginally) affecting the quality of results. However, there is still room for improvement.

The proper resource-driven design of KWS pipelines calls for an effective and fast accuracy-vs-latency multi-objective optimization process that cannot be the mere superimposition of independent minimization techniques applied at one stage or another. Doing so means overlooking the potential benefits of comprehensive and systematic optimization. Frontend and ConvNet are tightly coupled, and their hyperparameter settings mutually affect each other, thereby impacting the quality of the resulting KWS implementation.

This work investigates the potential savings brought by joint optimizations, exploring the interplay between frontend and ConvNet optimization. We show how the Pareto-dominant configurations are distributed in the accuracy-latency objective space and quantify the distance from the results returned by state-of-the-art optimizations that work on the two pipeline stages independently. To achieve this goal, we developed a design space exploration framework in which the most significant hyperparameters are discretized to meet the specifications of common embedded speech processing systems. The

979-8-3503-3761-7/23/$31.00 ©2023 IEEE

Authorized licensed use limited to: Universidad de Sevilla. Downloaded on August 04,2026 at 11:07:06 UTC from IEEE Xplore. Restrictions apply.

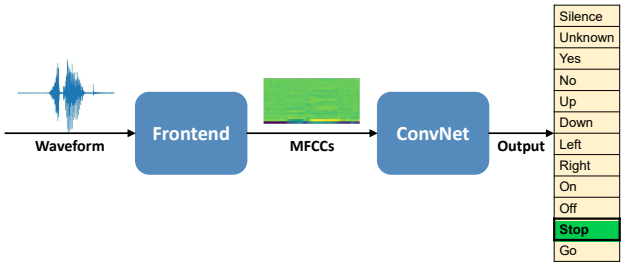


Fig. 1. Overview of the embedded KWS pipeline.

accuracy-vs-latency assessment is conducted using the Speech Command Dataset v0.02 [21], a popular dataset for KWS, and using as testbench a commercial embedded board, the Raspberry Pi 4 powered with an ARM Cortex-A72 processor.

The results obtained through our experimental setup show that joint optimization yields better accuracy-latency trade-offs than independent stage-wise optimizations. More specifically, the joint optimization approach provides a larger space of solutions with a higher number of Pareto-efficient configurations ranging from 92.11% accuracy at 3.1 ms to 96.15% accuracy at 20.08 ms, ensuring up to 1.8% higher accuracy and 27.36% faster processing compared to state-of-the-art approaches. The insights gained from the numerical analysis offer valuable guidance to designers of embedded KWS applications, helping configure the pipeline stages to achieve concurrent improvements in accuracy and latency.

II. Embedded Keyword Spotting Pipeline and Optimization Knobs

The standard two-stage inference pipeline for embedded KWS studied in this work is shown in Fig. 1. The figure refers to the 12-label KWS task that we used as a benchmark; however, the same pipeline works with any set and number of labels. As introduced in the previous section, the first stage is referred to as the frontend, which involves the computation of the Mel-Frequency Cepstral Coefficients (MFCCs). The second stage is the ConvNet, which processes the generated features to classify the possible keyword contained in the input audio. The following paragraphs provide a more detailed description of the two stages.

a) Frontend and its hyperparameters: The standard frontend dataflow for embedded KWS is depicted in Fig. 2. It takes a raw audio waveform as input and returns the MFCC features. The first step is the application of the Short-Time Fourier Transform (STFT), which splits the signal into frames of fixed length, defined by the *Frame Length* parameter, with partial overlap between consecutive frames. The overlap depends on the time shift between consecutive frames, the *Frame Step* parameter expressed as a fraction of the *Frame Length* (e.g., with a *Frame Length* of 40 ms and *Frame Step* 75%, the time shift is 30 ms). For each frame, a Hann window filter is applied to reduce the spectral leakage, and the Discrete Fourier Transform is computed on the filtered signal. From the resulting frequency spectrum, the spectrogram is obtained by computing the square norm of the complex values (i.e., the magnitude). The *Frame Length* and *Frame Step* hyperparameters affect the temporal resolution of the STFT, which in turn may alter the quality of the MFCCs features as well as the workload. Specifically, a shorter *Frame Length* enables a better evolution of the frequency components over time, hence higher quality, whereas a shorter *Frame Step* reduces the information loss due to Hann filtering, and hence higher quality. In contrast, the lower the values for these two hyperparameters, the larger the number of frames to process, resulting in a longer execution time.

The spectrogram is then processed with a set of triangular filter banks, known as Mel filters, which are spaced on the Mel scale. The Mel scale is a nonlinear scale that better represents how human ears perceive differences in pitch, placing more emphasis on lower frequency bands. The output of the Mel filter bank is passed through a logarithmic function to obtain the log-Mel spectrogram, mimicking how human hearing responds to sound intensity and perceives loudness on a logarithmic scale. The number of Mel filters (*# Mel Filters*) defines the frequency resolution, which increases with a larger number of filters. Higher resolutions improve the quality of the spectral content at the cost of a higher dimensionality of the feature vector and, hence, more computational overhead.

In the final stage, the log-Mel values are fed into a Discrete Cosine Transform (DCT) to extract MFCC features containing information about the spectral envelope of the speech signal. Such MFCC features have proven to be effective in capturing phonetic information for speech recognition tasks. During the DCT, the number of coefficients (*# MFCCs*) defines the cardinality of the output features. Only a limited *# MFCCs* is typically selected for the subsequent processing steps [22]. The first few coefficients capture information regarding the shape of the vocal tract, which is important for discriminating between different phonemes and sounds. The remaining coefficients represent the high-frequency harmonics of the signal, which are less important for speech recognition and can therefore be discarded. The higher the *# MFCCs*, the larger is the number of arithmetic operations during inference, which in turn reflects a longer processing time.

b) ConvNets and their hyperparameters: The most compact networks for embedded KWS consist of a chain of depthwise separable convolution operators [7], [19]. A depthwise separable convolution consists of two main steps: (1) depthwise convolution, where each input channel is convolved with a single filter; (2) pointwise convolution, where 1×1 convolutions combine the outputs generated by the depthwise convolution. This organization leads to an approximation of the standard matrix convolution and substantial arithmetic operation savings. For instance, a depthwise convolution with 3×3 kernels can reduce the number of operations by a factor

Authorized licensed use limited to: Universidad de Sevilla. Downloaded on August 04,2026 at 11:07:06 UTC from IEEE Xplore. Restrictions apply.

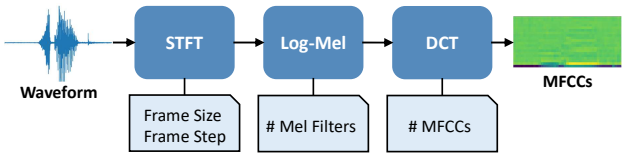


Fig. 2. Frontend dataflow for embedded KWS.

of 8×-9× compared to the standard implementation [23]. For this reason, depthwise separable convolution operators are typically the preferred option when dealing with resource-bounded devices. After the convolutional layers, the last two layers of the ConvNet consist of an *average pooling* function, which reduces the spatial resolution of the neural features to one, and a *softmax* classifier that computes the probability scores for each class.

Designers can customize the ConvNet template to achieve different accuracy-latency trade-offs. Intuitively, the available architectural knobs at hand are the number of stacked depthwise separable convolution layers (*# DS-Conv*) and the number of filters in each convolution (*# Filters*), both affecting the processing time proportionally.

III. Related Works on KWS Optimization

This section provides an overview of previous research on the optimization of KWS pipelines, with an emphasis on methods for the frontend and ConvNet stages.

A. Frontend optimizations

The study conducted in [8] investigated the impact of the frontend's hyperparameters tuning via grid-search exploration. Additionally, [9] proposed incorporating frontend hyperparameter search into a gradient descent optimization framework to automate the search process. These studies neglected the potential benefits of joint optimization and overlooked the interdependence between frontend and ConvNet configurations. Therefore, in our work, we perform the search for optimal solutions through a joint framework in which both frontend and ConvNet hyperparameters form a unique design space that enables the exploration of new dominant solutions.

Other studies have emphasized the importance of reducing the computational complexity of the frontend to reduce the overall pipeline latency. One option encompasses the use of fixed-point representation to reduce the arithmetic precision of MFCCs computations, for instance, using lower bit-widths, from 16-bit [10] to 2-bit [11]. These methods complement the frontend hyperparameter tuning and can potentially provide additional speedup, yet with an extra accuracy loss. It is also worth noting that fixed-point arithmetic brings performance gains if supported by a proper instruction set and dedicated processing units, which are often not available on off-the-shelf, general-purpose cores.

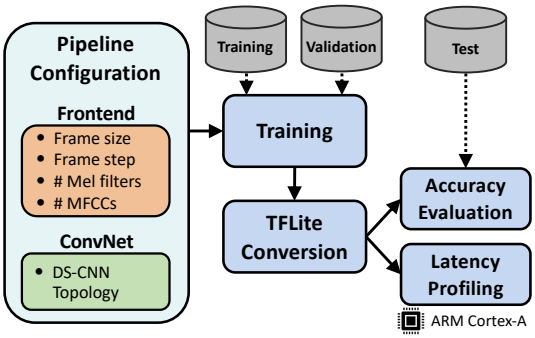


Fig. 3. Design Space Exploration Framework.

Another interesting research thread investigated the replacement of the MFCCs extraction stack with a learnable neural architecture. In [12], the authors proposed a frontend that uses one-dimensional convolutions trained to extract the most suitable feature representations. Alternative neural architectures were explored in [13], [14]. In both cases, the primary objective was to improve accuracy, with no consideration for latency. Replacing standard MFCCs with a neural network may result in 100× longer CPU time [24]. This suggests that a learnable frontend might be suited for GPU inference in the cloud rather than embedded inference on low-power devices deployed at the edge.

B. ConvNet optimizations

Many studies have focused on viable strategies to improve the classification accuracy of ConvNets deployed in KWS pipelines, with *residual learning*, *dilated convolutions*, and *temporal convolutions* being the most promising. Residual learning [25] introduces a series of shortcut connections between nonconsecutive layers that make deeper neural networks easier to train. The results reported in [16] show remarkable accuracy improvements. Dilated Convolutions enlarge the convolutional kernels by inserting gaps between their elements. This allows the same kernel to cover a wider portion of the input sample, resulting in a larger receptive field for the same number of parameters [26]. Temporal Convolution [27] is a 1D convolution applied to the temporal axis of the MFCCs, which treats the frequency values as distinct input channels. This approach reduces the number of computations while enlarging the receptive field of the network.

Another practical option to reduce the computational complexity of ConvNets consists of classical compression techniques such as quantization and pruning [28]. Both quantization and sparsification of the model weights and activations proved effective in reducing the latency of small-footprint ConvNets with a marginal loss in accuracy [17].

Authorized licensed use limited to: Universidad de Sevilla. Downloaded on August 04,2026 at 11:07:06 UTC from IEEE Xplore. Restrictions apply.

TABLE I
CONVNET DESIGN SPACE

	DS-CNN-S	DS-CNN-M	DS-CNN-L
# DS-Conv	5	5	6
# Filters	64	172	276

TABLE II
FRONTEND DESIGN SPACE.

Parameters	Values
Frame Size (ms)	20, 32, 40
Frame Step	50%, 75%, 100%
# Mel Filters	16, 28, 40
# MFCCs	8, 10, 12

IV. KWS DESIGN SPACE EXPLORATION FRAMEWORK

A. Overview

Fig. 3 illustrates a block diagram of the framework used for the Design Space Exploration (DSE) of the two-stage embedded KWS pipeline targeted in this work. The framework is fed with (i) the pipeline configuration defined through the set of tunable hyperparameters and (ii) the target dataset, split into *Training*, *Validation*, and *Test* sets. Each explored configuration undergoes training and conversion to TFLite format before deployment on the target ARM Cortex-A CPU, where classification accuracy and execution time are measured.

B. Dataset

For training and evaluation, we used the Speech Commands Dataset v0.02 [21], which contains 65k one-second-long recordings of 30 keywords pronounced by different speakers. Each recording is sampled at 16 kHz with a 16-bit resolution. Following the same procedure presented in [7], the ConvNets are trained to classify 10 keywords ("Yes", "No", "Up", "Down", "Left", "Right", "On", "Off", "Stop", "Go") out of the 30 available, and two additional classes: "Silence", which contains recordings with background noise, and "Unknown", which is populated with samples from the 20 excluded classes. The resulting dataset is split into a set for training (80% of all samples), validation (10%), and testing (10%).

C. Joint Design Space

In the frontend design space, we included the hyperparameters described in Sec. II-a, which have the greatest impact on the latency and accuracy of the KWS pipeline. We selected their values to ensure the exploration of configurations for practical use in speech applications for embedded systems. Specifically, we identified values that are more often used in commercial, state-of-the-art implementations. We refer to these values as the baseline. A parameters sweep is performed to drive the search toward implementations faster than the baseline. Table II shows the resulting search space.

Frame Size ranges from 20 ms to 40 ms, where the speech signal is assumed to be stationary [4], [7], [29], [30], i.e., its spectral contents are constant in time. *Frame Step* ranges from 50% (common in many implementations) to 100% (no overlap) with step 25%. *# Mel filters* ranges from 16 to 40, with an intermediate value of 28. 40 is one of the most commonly used in the literature [4], [7], [10], [19], and is the default in the most popular software libraries. *# MFCCs* ranges from 8 to 12, step 2. 12 and 10 are de-facto standards, but we also included 8 to explore a wider space.

For what concerns the design space of the ConvNet stage, we explored three implementations of the DS-CNN topology proposed in [7] (please refer to Sec. II-b) with increasing size: small (S), medium (M), large (L). The numerical details are presented in Table I.

D. Training

Each pipeline corresponding to a given hyperparameter configuration in the design space followed a supervised training using the TensorFlow framework. The training loop consists of 30000 iterations with a batch size of 100 and the Adam optimizer [31]. The learning rate was set to an initial value of $5\mathrm{e}{-4}$ and decayed by a factor of 0.2 every 10000 iterations. Moreover, we applied data augmentation in the form of added background noise and random time shifts. At the end of each training epoch, the cross-entropy loss was measured over the validation set and, if lower than the previous checkpoint, the model weights were saved.

E. TFLite Conversion

To deploy the trained pipelines in the ARM Cortex-A architecture, we leveraged the optimized routines available in TensorFlow.

For frontend acceleration, TensorFlow offers optimized implementations of common signal processing functions made available in the *tf.raw_ops* namespace through low-level access to primitives for tensor operations. We picked the *AudioSpectrogram* and *Mfcc* functions, which perform the computation of the STFT spectrogram and the MFCCs, respectively.

For ConvNet acceleration, we relied on TensorFlow Lite (TFLite), an inference engine for resource-constrained computing systems. TFLite exposes a Python API that bridges high-level model descriptions with low-level routines optimized for several hardware architectures.

When ported onto Cortex-A CPUs, the TFLite interpreter delegates the execution of the model operators to the XNNPACK library. The latter implements the most critical neural operators, like the depthwise separable convolution operators, written and optimized in assembly [32].

F. Accuracy Evaluation

The prediction quality is assessed on the test set. We adopted the *macro F1* score as the evaluation metric. Specifically,

Authorized licensed use limited to: Universidad de Sevilla. Downloaded on August 04,2026 at 11:07:06 UTC from IEEE Xplore. Restrictions apply.

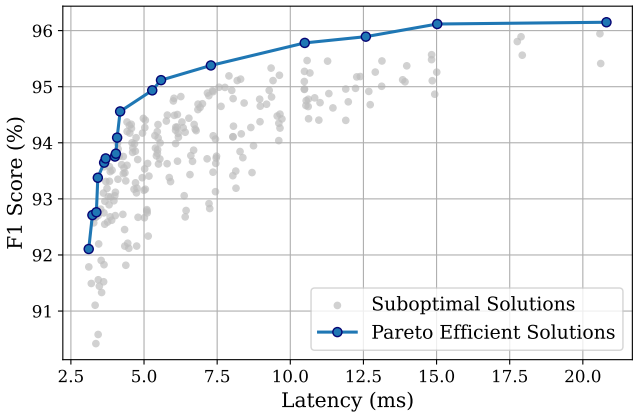


Fig. 4. F1 Score vs. Latency of the solutions returned by our DSE framework.

we first evaluated the F1 score for each class, calculated as the harmonic mean between precision and recall, and then computed the unweighted average of the per-class scores.

G. Latency Profiling

We ported each generated pipeline to a Raspberry Pi 4 (Model B), an off-the-shelf embedded board powered by the Broadcom BCM2711 System-on-Chip with a quad-core ARM Cortex-A72 CPU.

An in-house profiling tool packs random inputs and collects onboard latency statistics. The outcome is the CPU time taken by the whole pipeline (i.e., frontend and ConvNet) fed with a one-second-long input audio. We averaged the CPU time over 100 inference runs interleaved by a 2-second pause to avoid thermal throttling. A sequence of 20 warm-up runs initiates the actual profiling, mitigating potential delays due to setup routines for model initialization and memory allocation. All experiments were run with four-thread execution and maximum clock frequency (1.8 GHz).

H. Software environments

We used Ubuntu Server 20.04.5 LTS as the operating system, Python 3.9.5, and TensorFlow 2.11.0.

V. Experimental Results

Fig. 4 shows the F1 score and latency of the solutions explored by the DSE framework. Out of the 243 (3^5) configurations resulting from all possible combinations of the hyperparameters in Tables II and I, only 17 (blue dots) lie in the Pareto frontier. This preliminary analysis emphasized the importance of optimizing the hyperparameters of the KWS pipeline. Neglecting this aspect can lead to suboptimal implementations with negative effects on both accuracy and latency. To quantify the achievable optimization savings, we conducted a Pareto analysis of three representative strategies defined as follows:

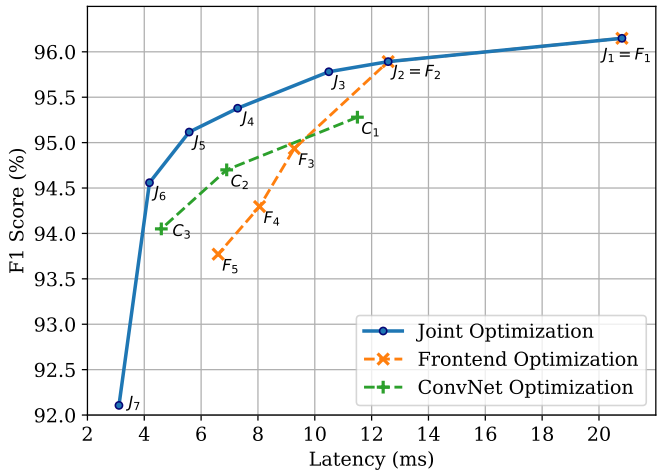


Fig. 5. Trade-off frontier comparison among Frontend, ConvNet, and Joint Optimization.

TABLE III
Overview of the solutions belonging to the Joint Optimization trade-off Frontier.

	Hyperparameters					Metrics	
	Frame Size (ms)	**Frame Shift**	**# Mel Filters**	**# MFCCs**	**DS-CNN**	**F1 Score**	**Latency** (ms)
J_1	20	50%	40	12	L	96.15%	20.8
J_2	40	50%	16	12	L	95.89%	12.6
J_3	20	50%	16	12	M	95.78%	10.5
J_4	20	75%	40	10	M	95.38%	7.3
J_5	20	50%	16	12	S	95.12%	5.6
J_6	32	50%	16	10	S	94.56%	4.2
J_7	32	100%	28	8	S	92.11%	3.1

a) Joint Optimization: considers the joint exploration of frontend parameters and ConvNet architectures. To ensure diversity among the alternative solutions, we applied ε-nondominated sorting [33] to the points reported in Fig 4. This methodology ensures that only a single Pareto-efficient point lies in a box with width ε_x and height ε_y. In our analysis, we set ε_x=1 ms and ε_y=0.1%. We annotated the resulting set of solutions as J_i, $i \in [1, 7]$ in Fig. 5 (• marker).

b) Frontend Optimization: that is the design space covers only frontend parameters. In particular, this approach is equivalent to exploring all combinations of the hyperparameters listed in Table II, while the ConvNet is fixed to a given architecture. For our analysis, we selected the DS-CNN-L architecture. Also in this case, we extracted a subset of the resulting solutions using ε-nondominated sorting with the same ε_x and ε_y as before, yielding the points F_j with $j \in [1, 5]$ in Fig. 5 ($\times$ markers).

c) ConvNet Optimization: that is the design space includes only alternative ConvNet architectures. Specifically, this approach consists of running the exploration with the same frontend for all three architectures listed in Table I. We set the frontend configuration of [7], i.e., with *Frame Length* 40 ms, *Frame Step* 50%, *#Mel Filters* 40, *# MFCCs* 10. The resulting

Authorized licensed use limited to: Universidad de Sevilla. Downloaded on August 04,2026 at 11:07:06 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
FRONTEND OPTIMIZATION VS JOINT OPTIMIZATION.

	F1 Score	Latency (ms)		F1 Gains	Latency Savings
F_1	96.15%	20.8	J_1	0%	0%
F_2	95.89%	12.6	J_2	0%	0%
F_3	94.93%	9.3	J_4	0.47%	27.36%
F_4	94.30%	8.1	J_4	1.15%	10.59%
F_5	93.43%	6.6	J_5	1.80%	17.73%

TABLE V
CONVNET OPTIMIZATION VS JOINT OPTIMIZATION.

	F1 Score	Latency (ms)		F1 Gains	Latency Savings
C_1	95.28%	11.5	J_2	0.53%	8.70%
C_2	94.70%	6.9	J_5	0.44%	18.84%
C_3	94.05%	4.6	J_6	0.54%	8.70%

solutions are represented as C_1, C_2, and C_3 in Fig. 5 (+ markers).

The trade-off frontier comparison in Fig. 5 shows that *Joint Optimization* outperforms stage-wise optimizations, ensuring improvements in both accuracy and latency. Table III lists the exact coordinates of the J_i points and their corresponding hyperparameter values. The reported data suggest that ConvNet architecture selection can be used as a coarse-grained knob to switch from more accurate (top part of the table) to faster (bottom part) solutions, whereas frontend tuning allows for a finer trade-off. For instance, selecting J_1 instead of J_2 leads to 39.5% faster execution, which can be achieved by simply increasing the Frame Size from $20\,\mathrm{ms}$ to $40\,\mathrm{ms}$ and reducing the # Mel Filters from 40 to 16. However, for further acceleration, it is better to select a smaller architecture rather than to set more compressed feature representations.

Moreover, the table reveals that the optimal frontend parameters vary depending on the ConvNet. Specifically, Pareto configurations with larger models (J_{1-4}) come with features with finer time resolution (i.e., smaller *Frame Size* and/or *Frame Step*), while the opposite holds for those with the smallest model (J_{5-7}). This finding demonstrates that the configuration of the frontend hyperparameters is tightly coupled with the ConvNet architecture and vice versa, and neglecting this coupling leads to suboptimal implementations.

We quantify the improvements brought by *Joint Optimization* in Tables IV and V, which compare our approach to *Frontend Optimization* and *Convent Optimization*, respectively. For each configuration reported on the left part of the table, the right part summarizes the accuracy gains and speedup achieved by the closest Pareto point belonging to the *Joint Optimization* curve. Table IV shows that *Joint Optimization* ensures up to 1.80% higher accuracy and 27.36% faster processing than *Frontend Optimization* (F_3 vs J_4). It should be noted that similar trends hold when conducting *Frontend Optimization* using a different ConvNet. Specifically, smaller architectures reduce latency but limit the maximum achievable accuracy, which is 95.78% (-0.38% compared to J_1) for DS-CNN-M and 95.12% (-1.07%) for DS-CNN-S. Table V highlights that *Joint Optimization* always achieves superior performance than *ConvNet Optimization*, with F1 gains ranging from 0.44% to 0.54% and latency savings from 8.70% to 18.84%.

Finally, *Joint Optimization* yields a larger number of configurations and covers a wider area in the trade-off space. In particular, moving from J_1 to J_7, the F1 score ranges from 96.15% to 92.11% and latency from $20.8\,\mathrm{ms}$ to $3.1\,\mathrm{ms}$, with $6.7\times$ faster processing at the cost of only 4.04% accuracy loss. Instead, from F_1 to F_6, the maximum acceleration is limited to $3.2\times$, whereas, from C_1 to C_3, the optimization space becomes even smaller, with an accuracy distance of 1.23% and a relative speedup of $2.5\times$. Besides being suboptimal, stage-wise optimizations also limit the possibility of reconfiguring the KWS pipeline based on the design requirements.

VI. CONCLUSION

In this work, we investigated a holistic approach for the optimization of embedded KWS pipelines, more specifically, the joint optimization of the frontend and ConvNet stages and we compared this approach with standard separate optimizations. The analysis conducted through our design space exploration framework yielded the following results: (i) joint optimization discovers solutions with performance superior to separate optimizations, with improvements up to 1.8% in accuracy and 27.36% in latency; (ii) within the design space exploration framework, the ConvNet architecture is a coarse-grained knob, while tuning frontend parameters allows for finer control in the accuracy vs. latency trade-off (iii) there exists a tight coupling between frontend and ConvNets settings, in particular, Pareto solutions with larger networks use more informative features while smaller networks use compressed and lossy features. Overall, our findings serve as guidelines for the implementation of embedded KWS applications and provide useful insights for future advancements in the optimization of deep-learning-based speech processing pipelines.

REFERENCES

[1] A. H. Michaely *et al.*, "Keyword spotting for google assistant using contextual speech recognition," in *2017 IEEE Automatic Speech Recognition and Understanding Workshop, ASRU 2017, Okinawa, Japan, December 16-20, 2017*. IEEE, 2017, pp. 272–278.

[2] I. López-Espejo, Z. Tan, and J. Jensen, "Improved external speaker-robust keyword spotting for hearing assistive devices," *IEEE ACM Trans. Audio Speech Lang. Process.*, vol. 28, pp. 1233–1247, 2020.

[3] I. Papp, Z. Saric, and N. Teslic, "Hands-free voice communication with TV," *IEEE Trans. Consumer Electron.*, vol. 57, no. 2, pp. 606–614, 2011.

[4] G. Chen, C. Parada, and G. Heigold, "Small-footprint keyword spotting using deep neural networks," in *IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP 2014, Florence, Italy, May 4-9, 2014*. IEEE, 2014, pp. 4087–4091.

[5] I. López-Espejo *et al.*, "Deep spoken keyword spotting: An overview," *IEEE Access*, vol. 10, pp. 4169–4199, 2022.

Authorized licensed use limited to: Universidad de Sevilla. Downloaded on August 04,2026 at 11:07:06 UTC from IEEE Xplore. Restrictions apply.

[6] R. Tang *et al.*, "An experimental analysis of the power consumption of convolutional neural networks for keyword spotting," in *2018 IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP 2018, Calgary, AB, Canada, April 15-20, 2018*. IEEE, 2018, pp. 5479–5483.

[7] Y. Zhang *et al.*, "Hello edge: Keyword spotting on microcontrollers," *CoRR*, vol. abs/1711.07128, 2017.

[8] W. Toussaint *et al.*, "Characterising the role of pre-processing parameters in audio-based embedded machine learning," in *SenSys '21: The 19th ACM Conference on Embedded Networked Sensor Systems, Coimbra, Portugal, November 15 - 17, 2021*. ACM, 2021, pp. 439–445.

[9] A. Zhao, K. Subramani, and P. Smaragdis, "Optimizing short-time fourier transform parameters via gradient descent," in *IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP 2021, Toronto, ON, Canada, June 6-11, 2021*. IEEE, 2021, pp. 736–740. [Online]. Available: https://doi.org/10.1109/ICASSP39728.2021.9413704

[10] M. Fariselli *et al.*, "Integer-only approximated MFCC for ultra-low power audio NN processing on multi-core mcus," in *3rd IEEE International Conference on Artificial Intelligence Circuits and Systems, AICAS 2021, Washington, DC, USA, June 6-9, 2021*. IEEE, 2021, pp. 1–4.

[11] A. Riviello and J. David, "Binary speech features for keyword spotting tasks," in *Interspeech 2019, 20th Annual Conference of the International Speech Communication Association, Graz, Austria, 15-19 September 2019*, G. Kubin and Z. Kacic, Eds. ISCA, 2019, pp. 3460–3464.

[12] K. W. Cheuk *et al.*, "nnaudio: An on-the-fly GPU audio to spectrogram conversion toolbox using 1d convolutional neural networks," *IEEE Access*, vol. 8, pp. 161 981–162 003, 2020.

[13] M. Ravanelli and Y. Bengio, "Speaker recognition from raw waveform with sincnet," in *2018 IEEE Spoken Language Technology Workshop, SLT 2018, Athens, Greece, December 18-21, 2018*. IEEE, pp. 1021–1028.

[14] N. Zeghidour *et al.*, "LEAF: A learnable frontend for audio classification," in *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*.

[15] T. N. Sainath and C. Parada, "Convolutional neural networks for small-footprint keyword spotting," in *INTERSPEECH 2015, 16th Annual Conference of the International Speech Communication Association, Dresden, Germany, September 6-10, 2015*. ISCA, 2015, pp. 1478–1482.

[16] R. Tang and J. Lin, "Deep residual learning for small-footprint keyword spotting," in *2018 IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP 2018, Calgary, AB, Canada, April 15-20, 2018*. IEEE, 2018, pp. 5484–5488.

[17] M. Grimaldi, V. Peluso, and A. Calimera, "Optimality assessment of memory-bounded convnets deployed on resource-constrained RISC cores," *IEEE Access*, vol. 7, pp. 152 599–152 611, 2019. [Online]. Available: https://doi.org/10.1109/ACCESS.2019.2948577

[18] B. Zhang *et al.*, "Autokws: Keyword spotting with differentiable architecture search," in *IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP 2021, Toronto, ON, Canada, June 6-11, 2021*. IEEE, 2021, pp. 2830–2834.

[19] C. R. Banbury *et al.*, "Micronets: Neural network architectures for deploying tinyml applications on commodity microcontrollers," in *Proceedings of Machine Learning and Systems 2021, MLSys 2021, virtual, April 5-9, 2021*, A. Smola, A. Dimakis, and I. Stoica, Eds. mlsys.org, 2021.

[20] D. Peter, W. Roth, and F. Pernkopf, "Resource-efficient dnns for keyword spotting using neural architecture search and quantization," in *25th International Conference on Pattern Recognition, ICPR 2020, Virtual Event / Milan, Italy, January 10-15, 2021*. IEEE, 2020, pp. 9273–9279.

[21] P. Warden, "Speech commands: A dataset for limited-vocabulary speech recognition," *CoRR*, vol. abs/1804.03209, 2018.

[22] V. Tiwari, "Mfcc and its applications in speaker recognition," 2010.

[23] A. G. Howard *et al.*, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," *CoRR*, vol. abs/1704.04861, 2017.

[24] J. Schlüter and G. Gutenbrunner, "Efficientleaf: A faster learnable audio frontend of questionable use," in *30th European Signal Processing Conference, EUSIPCO 2022, Belgrade, Serbia, August 29 - Sept. 2, 2022*. IEEE, 2022, pp. 205–208.

[25] K. He *et al.*, "Deep residual learning for image recognition," in *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016*. IEEE Computer Society, 2016, pp. 770–778.

[26] A. Coucke *et al.*, "Efficient keyword spotting using dilated convolutions and gating," in *IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP 2019, Brighton, United Kingdom, May 12-17, 2019*. IEEE, 2019, pp. 6351–6355.

[27] S. Choi *et al.*, "Temporal convolution for real-time keyword spotting on mobile devices," in *Interspeech 2019, 20th Annual Conference of the International Speech Communication Association, Graz, Austria, 15-19 September 2019*, G. Kubin and Z. Kacic, Eds. ISCA, 2019, pp. 3372–3376.

[28] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural network with pruning, trained quantization and huffman coding," in *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, Y. Bengio and Y. LeCun, Eds., 2016.

[29] G. Tucker *et al.*, "Model compression applied to small-footprint keyword spotting," in *Interspeech 2016, 17th Annual Conference of the International Speech Communication Association, San Francisco, CA, USA, September 8-12, 2016*, N. Morgan, Ed. ISCA, 2016, pp. 1878–1882.

[30] R. Alvarez and H. Park, "End-to-end streaming keyword spotting," in *IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP 2019, Brighton, United Kingdom, May 12-17, 2019*. IEEE, 2019, pp. 6336–6340.

[31] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, Y. Bengio and Y. LeCun, Eds., 2015.

[32] M. Dukhan, "The indirect convolution algorithm," *CoRR*, vol. abs/1907.02129, 2019. [Online]. Available: http://arxiv.org/abs/1907.02129

[33] M. Laumanns *et al.*, "Combining convergence and diversity in evolutionary multiobjective optimization," *Evol. Comput.*, vol. 10, no. 3, pp. 263–282, 2002.

Authorized licensed use limited to: Universidad de Sevilla. Downloaded on August 04,2026 at 11:07:06 UTC from IEEE Xplore. Restrictions apply.